End-to-End Execution Guide v3 (Updated)

Casting Defect Analysis Pipeline & REST API Serving

PHASE 1: PROJECT SETUP & DIRECTORIES

■ Step 1: Create & Enter Project Folder

```
mkdir DefectAnalyser
cd DefectAnalyser
```

■ Step 2: Create Training Folder

```
mkdir training
```

■ Step 3: Setup Data Folder & Kaggle Dataset

Place your downloaded Kaggle casting dataset matching this directory structure inside `data/`:

```
data/
└── casting_data/
    └── casting_data/
        ├── train/ (def_front / ok_front)
        └── test/ (def_front / ok_front)
```

PHASE 2: ENVIRONMENT INITIALIZATION

■ Step 4 & 5: Python 3.12 & Virtual Environment

```
py install 3.12
py -3.12 -m venv qualityenv
```

PHASE 3: ACTIVATION & DEPENDENCIES

■ Step 6, 7 & 8: Activation & TensorFlow Install

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
.\qualityenv\Scripts\Activate.ps1
pip install tensorflow
```

PHASE 4: CODE & EXECUTION

■ Step 9: Point VS Code Interpreter

Select Python 3.12.x (`qualityenv` : venv) in VS Code via `Ctrl + Shift + P` .

■ Step 10: Updated `training_model.py` Code

Create `training/training_model.py` with absolute pathing compatible with your Kaggle layout:

```
import os
import tensorflow as tf
from tensorflow.keras import layers, models

CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
BASE_DIR = os.path.dirname(CURRENT_DIR)
DATA_DIR = os.path.join(BASE_DIR, "data", "casting_data", "casting_data")
TRAIN_DIR = os.path.join(DATA_DIR, "train")
TEST_DIR = os.path.join(DATA_DIR, "test")
MODEL_STORE_DIR = os.path.join(BASE_DIR, "model_store")

IMG_SIZE = (512, 512)
BATCH_SIZE = 16
EPOCHS = 10

def run_training_pipeline():
    os.makedirs(MODEL_STORE_DIR, exist_ok=True)
    train_ds = tf.keras.utils.image_dataset_from_directory(
        TRAIN_DIR, image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=True
    )
    test_ds = tf.keras.utils.image_dataset_from_directory(
        TEST_DIR, image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=False
    )
    model = models.Sequential([
        layers.Input(shape=(512, 512, 3)),
        layers.Rescaling(1./255),
        layers.Conv2D(32, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    model.fit(train_ds, validation_data=test_ds, epochs=EPOCHS)

    model_path = os.path.join(MODEL_STORE_DIR, "casting_defect_classifier.keras")
    model.save(model_path)
    print(f"Model saved to: {model_path}")

if __name__ == "__main__":
    run_training_pipeline()
```

■ Step 11: Execute Script

```
python training/training_model.py
```

PHASE 5: MODEL DEPLOYMENT & SERVING (REST API)

■ Step 12: Create Serving Directory

```
mkdir serving
```

■ Step 13: Install API Dependencies

```
pip install fastapi uvicorn pillow numpy
```

■ Step 14: Updated `serving/api.py` Code (Direct File Upload Version)

Create `serving/api.py` to handle direct image uploads via FastAPI:

```
import os
import numpy as np
from fastapi import FastAPI, HTTPException, UploadFile, File
from pydantic import BaseModel
import tensorflow as tf
from PIL import Image
import io

app = FastAPI(
    title="Casting Defect Classification API",
    description="REST API for serving the casting defect classifier model with direct image upl
    version="1.1"
)

CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
BASE_DIR = os.path.dirname(CURRENT_DIR)
MODEL_PATH = os.path.join(BASE_DIR, "model_store", "casting_defect_classifier.keras")

print(f"Loading model from: {MODEL_PATH}")
if os.path.exists(MODEL_PATH):
    model = tf.keras.models.load_model(MODEL_PATH)
else:
    raise FileNotFoundError(f"Model file not found at {MODEL_PATH}. Please run training first."

IMG_SIZE = (512, 512)

class InferenceResponse(BaseModel):
    prediction: float
    result: str

@app.get("/")
def health_check():
    return {"status": "healthy", "model_loaded": True}

@app.post("/predict", response_model=InferenceResponse)
async def predict_defect(file: UploadFile = File(...)):
    try:
        contents = await file.read()
        image = Image.open(io.BytesIO(contents)).convert("RGB")
        image = image.resize(IMG_SIZE)

        img_array = np.array(image, dtype=np.float32)
        img_array = np.expand_dims(img_array, axis=0)

        prediction = model.predict(img_array)
        score = float(prediction[0][0])
        result_label = "defective" if score >= 0.5 else "ok"

        return InferenceResponse(
            prediction=score,
            result=result_label
        )
```

```
    except Exception as e:
        error_msg = str(e.__dict__) if hasattr(e, '__dict__') else str(e)
        raise HTTPException(status_code=400, detail=f"Error processing image: {error_msg}")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("api:app", host="0.0.0.0", port=8000, reload=True)
```

■ Step 15: Run the REST API

```
python serving/api.py
```

Access the interactive Swagger documentation in your browser at: **http://127.0.0.1:8000/docs** to upload test images directly via the file browser button (Choose File).